Summary: An Extreme Learning Machine-Based Method for Computational PDEs in Higher Dimensions

Yiran Wang and Suchuan Dong
Computer Methods in Applied Mechanics and Engineering, 2024

May 2, 2025

Abstract

This document summarizes the paper "An Extreme Learning Machine-Based Method for Computational PDEs in Higher Dimensions" by Wang and Dong (2024). The paper presents two effective randomized neural network methods for solving high-dimensional partial differential equations (PDEs): the Extreme Learning Machine (ELM) method and its variant combined with an approximate Theory of Functional Connections (ELM/A-TFC). Both methods address the curse of dimensionality by leveraging randomized neural networks where only output-layer parameters are trained. The paper demonstrates through extensive numerical experiments that these approaches can produce highly accurate solutions to high-dimensional PDEs with significantly better computational efficiency than traditional methods or physics-informed neural networks (PINNs). The methods show exponential convergence with respect to the number of trainable parameters and boundary collocation points, and can reach accuracies close to machine precision for lower-dimensional problems.

1 Introduction

High-dimensional partial differential equations (PDEs) arise in many fields including physics, biology, and finance. Traditional numerical methods such as finite difference, finite element, and spectral methods face severe challenges in high dimensions due to the curse of dimensionality, where computational complexity grows exponentially with increasing problem dimension.

While deep neural networks (DNNs) have emerged as a promising approach for solving high-dimensional PDEs, most DNN-based methods require training all network parameters, which is computationally expensive and often difficult to optimize. The paper presents two methods based on randomized neural networks that significantly reduce computational cost while maintaining high accuracy:

1. **Extreme Learning Machine (ELM) method**: The solution is represented by a randomized neural network where hidden-layer parameters are randomly assigned and fixed, and only output-layer parameters are trained.

2. **ELM with Approximate Theory of Functional Connections (ELM/A-TFC)**: This combines ELM with a simplified variant of TFC to systematically enforce boundary conditions while avoiding exponential growth in computational complexity.

These methods are motivated by theoretical results showing that ELM-type networks can effectively approximate high-dimensional functions with a convergence rate independent of dimension in expectation.

2 Problem Statement

The methods address boundary value problems of the following form:

$$\mathcal{L}u(x) + \mu\mathcal{N}(u(x)) = Q(x), \quad x \in \Omega \tag{1}$$

$$\mathcal{B}u(x) = H(x), \quad x \in \partial\Omega \tag{2}$$

where:

- $\Omega = \Omega_1 \times \Omega_2 \times \cdots \times \Omega_d$ is a domain in $\mathbb{R}^d$ with boundary $\partial\Omega$
- $\Omega_i = [a_i, b_i]$ for constants a_i and b_i $(1 \leq i \leq d)$
- $\mathcal{L}$ and $\mathcal{N}$ are linear and nonlinear differential operators, respectively
- $\mathcal{B}$ is a linear differential or algebraic operator representing boundary conditions
- $u(x) \in \mathbb{R}$ is the unknown field to be computed
- Q and H are given functions, and μ is a constant

For time-dependent problems, time t is treated as an additional dimension, making it a $(d+1)$-dimensional problem.

3 Theoretical Foundation

A key theoretical result motivating these methods is that ELM-type randomized neural networks can overcome the curse of dimensionality in function approximation.

For Lipschitz functions, the expected L^2 error for approximation by randomized neural networks decays as $O(1/\sqrt{n})$ with the number of random basis functions n, independent of the dimension d. This contrasts with deterministic basis methods, where the error decays as $O(n^{-1/d})$ and suffers from the curse of dimensionality.

Specifically, consider a Lipschitz continuous function $f \in C(I^d)$ where $I^d = [0,1]^d$. Using a randomized neural network:

$$f_{\omega_n}(x) = \sum_{j=1}^{n} a_j g(w_j \cdot x + b_j) \tag{3}$$

where w_j and b_j are randomly assigned and fixed, while only a_j are trained. The theorem states:

$$E \int_K |f(x) - f_{\omega_n}(x)|^2 dx \leq \frac{C_{f,g,\Omega,\alpha,\beta,d}}{n} \tag{4}$$

where $C_{f,g,\Omega,\alpha,\beta,d}$ is independent of n. This indicates that the approximation error decreases as $O(1/\sqrt{n})$ regardless of dimension.

4 The ELM Method

4.1 Algorithm Description

The ELM method represents the solution $u(x)$ to a PDE as:

$$u(x) = \sum_{j=1}^{N_u} \phi_j V_j(x) = \mathbf{V}(x)\mathbf{\Phi}, \quad x \in \Omega \tag{5}$$

where:

- $V_j(x)$ are the outputs from the last hidden layer of a feed-forward neural network
- ϕ_j are the output-layer coefficients (trainable parameters)
- $\mathbf{V}(x) = (V_1(x), \ldots, V_{N_u}(x))$ is fixed once hidden-layer coefficients are randomly assigned
- $\mathbf{\Phi} = (\phi_1, \ldots, \phi_{N_u})^T$ are the parameters to be trained

The residual function of the PDE system is:

$$\mathbf{R}(\mathbf{\Phi}) = \begin{bmatrix} \mathbf{R}_{\text{pde}}(\mathbf{\Phi}) \\ \mathbf{R}_{\text{bc}}(\mathbf{\Phi}) \end{bmatrix} = \begin{bmatrix} \mathbf{V}(\mathbf{x})\mathbf{\Phi} + \mu\mathcal{N}(\mathbf{V}(\mathbf{x})\mathbf{\Phi}) - Q(\mathbf{x}) \\ \mathbf{V}(\mathbf{y})\mathbf{\Phi} - H(\mathbf{y}) \end{bmatrix} \tag{6}$$

The algorithm proceeds as follows:

Algorithm 1 ELM Method for High-Dimensional PDEs

1: Set up a feedforward neural network with architecture $[d, M, 1]$ where d is the problem dimension
2: Randomly assign hidden-layer parameters from the interval $[-R_m, R_m]$ and fix them
3: Generate N_{in} random collocation points in the interior of Ω
4: Generate N_{bc} random collocation points on each hyperface of $\partial\Omega$
5: Enforce the residual function to be zero on all collocation points, forming an algebraic system
6: Solve for $\mathbf{\Phi}$ using linear least squares (if $\mu = 0$) or nonlinear least squares (if $\mu \neq 0$)
7: Compute the solution as $u(x) = \mathbf{V}(x)\mathbf{\Phi}$

For nonlinear problems ($\mu \neq 0$), the Jacobian matrix for the nonlinear least squares method is:

$$\mathbf{J}(\mathbf{\Phi}) = \begin{bmatrix} \mathbf{V}(\mathbf{x}) + \mu\mathcal{N}'(\mathbf{V}(\mathbf{x})\mathbf{\Phi})\mathbf{V}(\mathbf{x}) \\ \mathbf{V}(\mathbf{y}) \end{bmatrix} \tag{7}$$

4.2 Domain Decomposition: Local ELM

For problems with sharp gradients or local features, a domain decomposition approach called local ELM (locELM) is employed:

- Domain Ω is decomposed along a maximum of ℓ directions (where $\ell \ll d$, typically $\ell = 2$)
- Each subdomain Ω_i uses its own ELM neural network
- Continuity conditions are enforced across shared subdomain boundaries
- The solution in each subdomain is $u_i(x) = \mathbf{V}_i(x)\mathbf{\Phi}_i$, $x \in \Omega_i$

5 The ELM/A-TFC Method

5.1 Approximate Theory of Functional Connections (A-TFC)

The Theory of Functional Connections (TFC) provides a systematic approach to enforce boundary conditions through constrained expressions, but the number of terms grows exponentially with dimension.

The A-TFC approach introduces a hierarchical truncation to avoid this exponential growth:

For a function $u(x)$ satisfying $u|_{\partial\Omega} = C(x)$, the A-TFC constrained expression is:

$$u(x) = g(x) - \mathcal{A}_\Omega g(x) + \mathcal{A}_\Omega C(x) \tag{8}$$

where:

- $g(x)$ is a free function to be determined
- $\mathcal{A}_\Omega$ is an approximation of the TFC operator that retains only dominant terms
- $\mathcal{A}_\Omega f(x) = \mathcal{A}^1_\Omega f(x)$ instead of the full TFC expression $\mathcal{A}_\Omega f(x) = \sum_{i=1}^{d} (-1)^{i-1} \mathcal{A}^i_\Omega f(x)$

The first-order term $\mathcal{A}^1_\Omega f(x)$ for a function f defined on $\partial\Omega$ is:

$$\mathcal{A}^1_\Omega f(x) = \sum_{i=1}^{d} \left[\frac{b - x_i}{b - a} f(x)|_{x_i=a} + \frac{x_i - a}{b - a} f(x)|_{x_i=b} \right] \tag{9}$$

5.2 Algorithm Description

The ELM/A-TFC method represents the free function $g(x)$ by an ELM neural network:

$$g(x) = \sum_{j=1}^{N_g} \phi_j V_j(x) = \mathbf{V}(x)\mathbf{\Phi} \tag{10}$$

The PDE system is transformed to:

$$g(x) - \mathcal{A}_\Omega g(x) + \mu\mathcal{N}(\mathcal{A}_\Omega H(x) + g(x) - \mathcal{A}_\Omega g(x)) = Q(x) - \mathcal{A}_\Omega H(x), \quad x \in \Omega \tag{11}$$

$$\tilde{\mathcal{A}}_\Omega g(x) = \tilde{\mathcal{A}}_\Omega H(x), \quad x \in \partial\Omega \tag{12}$$

The algorithm is similar to the ELM method, but with the residual function based on the transformed system:

Algorithm 2 ELM/A-TFC Method for High-Dimensional PDEs

1: Set up a feedforward neural network with architecture $[d, M, 1]$ for representing $g(x)$
2: Randomly assign hidden-layer parameters from $[-R_m, R_m]$ and fix them
3: Generate random collocation points in the interior and on the boundary of Ω
4: Form the algebraic system based on the A-TFC transformed equations
5: Solve for $\mathbf{\Phi}$ using linear or nonlinear least squares
6: Compute $g(x) = \mathbf{V}(x)\mathbf{\Phi}$
7: Compute the solution as $u(x) = g(x) - \mathcal{A}_\Omega g(x) + \mathcal{A}_\Omega H(x)$

6 Numerical Experiments

6.1 Test Problems

The methods were tested on several high-dimensional PDEs:

- Poisson equation (dimensions up to 15)
- Nonlinear Poisson equation (dimensions up to 9)
- Advection-diffusion equation (dimensions up to 10)
- Korteweg-de Vries (KdV) equation (dimensions up to 10)
- Heat equation (dimensions up to 7)

6.2 Implementation Details

- Both methods used single hidden-layer neural networks with tanh activation function
- Hidden-layer coefficients were randomly assigned from $[-R_m, R_m]$ where R_m decreases with increasing dimension
- Random collocation points were used instead of grid-based points to avoid the curse of dimensionality
- Errors were computed on a separate set of test points, characterized by $(N_{bc}^{(v)}, N_{in}^{(v)}) = (100, 7000)$
- Linear least squares was used for linear problems, and nonlinear least squares with perturbations (NLLSQ-perturb) for nonlinear problems

6.3 Key Results

6.3.1 Effect of Training Parameters

For all test problems, the errors decreased quasi-exponentially with increasing number of training parameters until reaching a saturation level. The stagnant error level increased with dimension:

- For $d = 3$: errors reached 10^{-10} to 10^{-11}
- For $d = 5$: errors reached 10^{-8} to 10^{-9}
- For $d = 7 - 10$: errors reached 10^{-5} to 10^{-6}
- For $d = 15$: errors reached 10^{-5}

6.3.2 Effect of Collocation Points

Both methods showed:

- Strong dependence on boundary collocation points: errors decreased exponentially with increasing N_{bc} before saturation
- Minimal dependence on interior collocation points: N_{in} had little effect on accuracy, especially in higher dimensions

This reflects the increasing importance of boundaries in high dimensions, where the surface-to-volume ratio grows with dimension.

6.3.3 Comparison of ELM and ELM/A-TFC

- Both methods produced comparable accuracy
- ELM/A-TFC was slightly more accurate in lower dimensions, where A-TFC better approximates full TFC
- ELM required less computational effort and cost than ELM/A-TFC

6.3.4 Comparison with PINN

Both methods significantly outperformed classical PINN:

For the Poisson equation:

- At $d = 5$: PINN achieved errors of 10^{-2} to 10^{-3} in 552 seconds; ELM achieved 10^{-8} to 10^{-10} in 3.5 seconds
- At $d = 9$: PINN achieved errors of 10^{-1} in 964 seconds; ELM achieved 10^{-5} to 10^{-6} in 14.6 seconds

For the nonlinear Poisson equation:

- At $d = 5$: PINN achieved errors of 10^{-3} to 10^{-4} in 2526 seconds; ELM achieved 10^{-9} in 112.5 seconds
- At $d = 9$: PINN achieved errors of 10^{-2} in 5036 seconds; ELM achieved 10^{-4} to 10^{-5} in 678.8 seconds

7 Implications and Significance

7.1 Computational Advantages

The ELM-based methods provide several significant advantages:

- **Dimensionality**: They effectively overcome the curse of dimensionality that plagues traditional methods
- **Accuracy**: They can achieve near machine precision for lower dimensions and maintain good accuracy in higher dimensions
- **Efficiency**: Training is significantly faster than PINN methods (often by 2-3 orders of magnitude)
- **Convergence**: They exhibit exponential convergence with respect to the number of trainable parameters and boundary collocation points

7.2 Practical Considerations

Important practical findings include:

- The randomization parameter R_m for generating hidden-layer coefficients should decrease with increasing dimension
- Boundary collocation points are more important than interior points in high dimensions
- A small number of interior collocation points typically suffices in higher dimensions
- Domain decomposition (locELM) is beneficial for problems with sharp gradients or local features

8 Conclusion

The ELM and ELM/A-TFC methods represent significant advances in solving high-dimensional PDEs, combining theoretical rigor with practical efficiency. They effectively overcome the curse of dimensionality by leveraging the universal approximation property of randomized neural networks where error convergence is independent of dimension in expectation.

These methods demonstrate that it is possible to achieve high accuracy for high-dimensional PDEs with much lower computational cost than traditional methods or fully-trained neural networks like PINN. This makes them promising tools for scientific machine learning and computational mathematics, particularly for problems in physics, biology, and finance that involve high-dimensional PDEs.